

# **Co-Evolutionary Reinforcement Learning for Robust Detection of AI-Generated Text under Adaptive Adversarial Attacks**

*A B. Tech Project Report Submitted  
in Partial Fulfillment of the Requirements  
for the Degree of*

**Bachelor of Technology**

*by*

**Himanshu Kumar**  
(220101046)

*under the guidance of*

**Prof. Sanasam Ranbir Singh**



to the

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING  
INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI  
GUWAHATI - 781039, ASSAM**

# CERTIFICATE

*This is to certify that the work contained in this thesis entitled “**Co-Evolutionary Reinforcement Learning for Robust Detection of AI-Generated Text under Adaptive Adversarial Attacks**” is a bonafide work of **Himanshu Kumar (Roll No. 220101046)**, carried out in the Department of Computer Science and Engineering, Indian Institute of Technology Guwahati under my supervision and that it has not been submitted elsewhere for a degree.*

Supervisor: **Prof. Sanasam Ranbir Singh**

Assistant/Associate

Professor,

April, 2026

Engineering,

Guwahati.

Department of Computer Science &

Indian Institute of Technology Guwahati, Assam.

# Contents

|                                                        |            |
|--------------------------------------------------------|------------|
| <b>List of Figures</b>                                 | <b>iii</b> |
| <b>List of Tables</b>                                  | <b>iv</b>  |
| <b>1 Introduction</b>                                  | <b>1</b>   |
| 1.1 Adversarial Attacks on AI-generated Text . . . . . | 2          |
| 1.2 Detecting AI-generated Text . . . . .              | 2          |
| 1.3 Motivation and Objective . . . . .                 | 3          |
| <b>2 Literature Survey</b>                             | <b>5</b>   |
| 2.1 Datasets . . . . .                                 | 5          |
| 2.1.1 English Language Datasets . . . . .              | 6          |
| 2.1.2 Other Language Datasets . . . . .                | 6          |
| 2.2 Existing Detection Methods . . . . .               | 7          |
| 2.2.1 Zero-shot Methods . . . . .                      | 7          |
| 2.2.2 Fine-tuned Models . . . . .                      | 8          |
| 2.2.3 Deep Learning and Fusion Models . . . . .        | 8          |
| 2.2.4 Adversarial Learning Methods . . . . .           | 9          |
| 2.2.5 Summary and Limitations . . . . .                | 9          |
| <b>3 Methodology</b>                                   | <b>11</b>  |
| 3.1 System Overview . . . . .                          | 11         |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| 3.2      | Red Agent: Reinforcement Learning-based Attacker . . . . . | 12        |
| 3.2.1    | Policy Optimization using PPO . . . . .                    | 13        |
| 3.2.2    | GRPO-based Reward Stabilization . . . . .                  | 14        |
| 3.3      | Blue Agent: AI-text Detector . . . . .                     | 15        |
| 3.3.1    | Loss Function . . . . .                                    | 15        |
| 3.3.2    | Adversarial Update Strategy . . . . .                      | 16        |
| 3.4      | Diversity Buffer . . . . .                                 | 17        |
| 3.5      | Co-Evolutionary Training Process . . . . .                 | 18        |
| <b>4</b> | <b>Experimental Details, Results and Analysis</b>          | <b>20</b> |
| 4.1      | Experimental Details . . . . .                             | 20        |
| 4.1.1    | Datasets . . . . .                                         | 20        |
| 4.1.2    | Model Configuration . . . . .                              | 21        |
| 4.1.3    | Baselines . . . . .                                        | 25        |
| 4.1.4    | Hyperparameter Settings . . . . .                          | 26        |
| 4.2      | Results and Analysis . . . . .                             | 28        |
| 4.2.1    | Phase 1: Baseline Detector Analysis . . . . .              | 28        |
| 4.2.2    | Phase 2: COBRA Results and Comparison . . . . .            | 32        |
| <b>5</b> | <b>Conclusion and Future Work</b>                          | <b>37</b> |
| 5.1      | Conclusion . . . . .                                       | 37        |
| 5.2      | Limitations . . . . .                                      | 38        |
| 5.3      | Future Work . . . . .                                      | 39        |
|          | <b>References</b>                                          | <b>41</b> |

# List of Figures

|     |                                                                                                                                                                                                                                                                                                                                                                        |    |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.1 | Detailed training mechanism of COBRA. The Red Agent rewrites AI-generated text and receives a reward signal from the Blue Agent reflecting evasion success. Both agents are updated iteratively using PPO/GRPO optimization, driving progressive adversarial co-evolution. . . . .                                                                                     | 22 |
| 4.2 | Unified architecture of the COBRA framework. The Red Agent generates adversarial rewrites of AI-generated text, which are evaluated by the Blue Agent. Successful adversarial samples are stored in the Diversity Buffer and replayed during subsequent training iterations, enabling progressive improvement of both agents in a closed co-evolutionary loop. . . . . | 23 |
| 4.3 | Seen vs. Zero-shot performance comparison for COBRA-GRPO across all five evaluation datasets. Blue = Seen; Orange = Zero-shot. . . . .                                                                                                                                                                                                                                 | 36 |
| 4.4 | Zero-shot performance comparison between COBRA-PPO and COBRA-GRPO across all five evaluation datasets. Blue = PPO; Orange = GRPO. . . . .                                                                                                                                                                                                                              | 36 |

# List of Tables

|     |                                                                                                                                                                                                                               |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.1 | Model configuration for COBRA across language settings. . . . .                                                                                                                                                               | 21 |
| 4.2 | Hyperparameter settings for COBRA training. . . . .                                                                                                                                                                           | 27 |
| 4.3 | Feature comparison between human-written and AI-generated text for selected LLMs (English Dataset, Part 1). Values represent per-sample means. . . . .                                                                        | 29 |
| 4.4 | Feature comparison between human-written and AI-generated text for selected LLMs (English Dataset, Part 2). Values represent per-sample means. . . . .                                                                        | 30 |
| 4.5 | Feature comparison between human-written and AI-generated text across three Indian languages. H = Human, AI = AI-generated. . . . .                                                                                           | 31 |
| 4.6 | Performance of existing AI-text detectors on Indian language datasets. Results are macro-averaged across all ten Indian languages. Best result per metric in <b>bold</b> . . . . .                                            | 31 |
| 4.7 | Comparison of COBRA with baseline detectors. Results report F1 (Macro) on each dataset and average FPR across all datasets. † = multilingual configuration for CT2_AG_hi_OUR. Best result per column in <b>bold</b> . . . . . | 32 |
| 4.8 | COBRA performance on seen datasets. † denotes multilingual configuration for CT2_AG_hi_OUR. Best result per dataset in <b>bold</b> . . . . .                                                                                  | 34 |
| 4.9 | COBRA zero-shot performance across datasets. † denotes multilingual configuration for CT2_AG_hi_OUR. Best result per dataset in <b>bold</b> . . . . .                                                                         | 35 |

# Chapter 1

## Introduction

AI-generated text refers to content produced by Large Language Models (LLMs) such as GPT [Ope23], Gemini [Dee24], and Mistral [JSM<sup>+</sup>23], which are trained on large-scale human-written corpora. These models have achieved remarkable fluency, coherence, and contextual understanding, making their generated text almost indistinguishable from human-written content. As demonstrated in *Language Models are Few-Shot Learners* [BMR<sup>+</sup>20], modern LLMs are capable of generating high-quality text across a wide range of domains.

The rapid advancement of such models has introduced significant challenges in maintaining authenticity and trust. Unlike traditional plagiarism, AI-generated text is novel and does not directly copy existing content, making it difficult for conventional plagiarism detection systems to identify misuse. This has raised serious concerns in academia, journalism, and online content platforms.

Recent studies [WYZ<sup>+</sup>23a] highlight that detecting AI-generated text is becoming increasingly difficult due to the continuous improvement in generation quality and accessibility of AI tools like ChatGPT [Ope23] and GitHub Copilot. Users can easily generate essays, reports, and articles, making it challenging to differentiate between human-written

and AI-generated content.

## 1.1 Adversarial Attacks on AI-generated Text

A major challenge in AI-text detection is its vulnerability to adversarial attacks. Adversarial attacks refer to deliberate modifications applied to AI-generated text to evade detection systems.

As discussed in [WFH<sup>+</sup>24], simple transformations such as paraphrasing, word substitution, grammatical errors, and sentence restructuring can significantly degrade detection performance. These attacks exploit the fact that most detection methods rely on statistical and linguistic features such as perplexity, burstiness, and token distribution.

Paraphrasing tools such as QuillBot [Qui23] and Grammarly can rewrite AI-generated content while preserving semantic meaning, making it more human-like. Similarly, style transfer techniques modify tone, sentence structure, and writing patterns to mimic human writing [KSK<sup>+</sup>24].

These adversarial modifications disrupt the underlying statistical patterns used by detectors, making detection significantly more difficult. As a result, even minor modifications can allow AI-generated content to bypass existing detection systems [WFH<sup>+</sup>24], highlighting the need for more robust approaches.

## 1.2 Detecting AI-generated Text

Various methods have been proposed for detecting AI-generated text, including statistical, supervised, and adversarial approaches.

Statistical and zero-shot methods such as DetectGPT [M<sup>+</sup>23] and DetectLLM [SZWN23] rely on intrinsic properties of language models such as log-probability and token rankings. These approaches do not require labeled data but are computationally expensive and sensitive to paraphrasing.



Supervised methods involve fine-tuning pre-trained models such as RoBERTa [L<sup>+</sup>19]. The study [LZZ<sup>+</sup>23] demonstrates that fine-tuned models can achieve high accuracy in controlled settings. However, these models often suffer from overfitting and fail to generalize across unseen datasets and new language models.

Watermarking-based approaches [KGW<sup>+</sup>23] embed hidden patterns in generated text, enabling detection at inference time. However, these methods require access to generation mechanisms and are not robust against paraphrasing.

More advanced approaches such as RADAR [HCH23a] and OUTFOX [KKO24a] introduce adversarial training frameworks, where a detector is trained alongside an attacker. These methods improve robustness but are still limited by fixed or heuristic adversarial strategies.

### 1.3 Motivation and Objective

The widespread availability of generative AI tools has made it increasingly easy to produce high-quality text with minimal effort. This poses serious challenges to academic integrity, originality, and fairness.

Existing detection systems fail when AI-generated text is modified using paraphrasing or minor edits. Even advanced adversarial approaches such as RADAR [HCH23a] rely on predefined paraphrasing strategies and do not fully capture adaptive adversarial behavior.

This motivates the need for a more robust and adaptive detection framework that can handle dynamic adversarial scenarios and generalize across different datasets and language models.

#### **Problem Statement:**

Current AI-text detection models are not sufficiently robust to adversarial modifications such as paraphrasing, word substitution, and stylistic changes. These modifications significantly reduce detection accuracy, allowing AI-generated content to evade detection [WFH<sup>+</sup>24, M<sup>+</sup>23].

**Objectives:**

- To analyze the limitations of existing AI-text detection methods under adversarial conditions [WFH<sup>+</sup>24].
- To evaluate detection performance across multiple datasets and large language models [G<sup>+</sup>23, D<sup>+</sup>24a, L<sup>+</sup>24a, W<sup>+</sup>24].
- To design a co-evolutionary adversarial framework (COBRA) for robust AI-text detection.
- To incorporate reinforcement learning-based adversarial generation using Proximal Policy Optimization (PPO).
- To develop a model capable of strong zero-shot generalization across unseen LLMs [Ope23, Dee24, JSM<sup>+</sup>23, Coh24].
- To validate the effectiveness of the proposed approach through extensive experimentation.

# Chapter 2

## Literature Survey

This chapter reviews existing datasets and detection approaches for AI-generated text, and analyzes their robustness under adversarial conditions.

### 2.1 Datasets

A number of benchmark datasets have been created to support research in AI-generated text detection across different domains, languages, and LLM models. Besides English, some datasets also cover other languages like Urdu [AAKW25], Korean [PKKH25], Spanish [SGFS<sup>+</sup>23], Russian [SMC<sup>+</sup>22a], and Hindi [KRC<sup>+</sup>24b]. There are also shared tasks that support progress in this area, such as SemEval-2024 Task 8 [WMI<sup>+</sup>24a] and the multilingual GenAI content detection challenge [WSM<sup>+</sup>25].

Recently, several strong multilingual benchmarks have been introduced. These include M4GT-Bench [WMI<sup>+</sup>24b], which covers languages like Russian, Chinese, Urdu, Indonesian, and Arabic; MULTITuDE [MMU<sup>+</sup>23], which spans 11 languages including Dutch, Arabic, Chinese, Portuguese, Russian, Ukrainian, German, and Catalan; and MultiSocial [MKMS24], which expands multilingual coverage even further.

### 2.1.1 English Language Datasets

HC3 (Human–ChatGPT Comparison Corpus) [GZW<sup>+</sup>23] is one of the earliest open-source datasets developed to compare human-written with ChatGPT-generated responses. It was constructed by collecting answers from both humans and ChatGPT for the same set of questions and is available in both English and Chinese, making it valuable for multilingual studies.

Building upon HC3, the enhanced HC3-Plus [SWMH23] extends the original dataset by adding more diverse prompts, writing styles, and responses generated from newer LLMs, providing a stronger and more up-to-date resource for training and evaluating detectors.

M4GT-Bench [WMI<sup>+</sup>24b] is a multilingual dataset designed for AI-generated text detection. It includes outputs from multiple LLMs across various formats such as summaries, essays, and explanations, and spans several languages beyond English. Its multi-model and multi-format structure makes it a valuable resource for evaluating detector robustness in diverse and realistic settings.

MAGE [L<sup>+</sup>24b] provides realistic “in-the-wild” AI-generated samples drawn from actual user interactions, making it particularly useful for testing detectors under noisy, real-world conditions.

RAID [D<sup>+</sup>24b] focuses on text that has been deliberately edited or paraphrased, helping evaluate how robust a detector is when someone attempts to conceal AI-generated content. OUTFOX [KKO24b] similarly contains adversarially crafted examples specifically designed to fool detectors.

### 2.1.2 Other Language Datasets

RuATD [SMC<sup>+</sup>22b] is a Russian dataset designed for AI-generated text detection, providing paired human and machine-produced samples for evaluation on Cyrillic script and Russian linguistic patterns.

The Arabic subset of MULTITuDE [MMU<sup>+</sup>23] contains human-written and LLM-

generated Arabic text across several domains, offering insights into detecting AI-generated content in Semitic languages and non-Latin scripts.

CT-2 (Counter Turing Test – Hindi) [KRC<sup>+</sup>24a] is a dedicated Hindi dataset comprising both human and AI-generated text focused on commonly used topics, providing an important resource for evaluating detection methods on Devanagari-script languages.

Despite this progress, there is still no comprehensive publicly available dataset for AI-generated text detection that covers the majority of Indian languages, representing a clear gap in existing research.

## 2.2 Existing Detection Methods

Detection approaches for AI-generated text can be broadly categorized into zero-shot statistical methods, supervised fine-tuning approaches, deep learning models, and adversarial learning frameworks.

### 2.2.1 Zero-shot Methods

Zero-shot detection methods identify AI-generated text without requiring labeled training data by exploiting intrinsic statistical properties of language models.

detectgpt propose DetectGPT, based on the observation that machine-generated text lies in regions of negative curvature of the log-probability distribution. The method perturbs input text and compares probability differences to classify content as AI-generated. However, it requires access to model probabilities and is computationally expensive.

detectllm introduce two zero-shot methods: DetectLLM-LLR and DetectLLM-NPR, which leverage log-rank information alongside log-likelihood. The Log-Likelihood Log-Rank Ratio (LLR) combines both signals for stronger detection, while Normalized Perturbation Rank (NPR) exploits the sensitivity of AI-generated text to small perturbations. Though more accurate than LLR, NPR is slower due to the need to generate multiple perturbed versions.

GLTR [GSR19] is a statistical visualization tool that checks how likely each word is according to a language model, exploiting the tendency of AI models to select high-probability tokens. It computes word probability, rank, and entropy, and was shown to improve human detection accuracy significantly in user studies.

Binoculars [HSC<sup>+</sup>24] is a strong zero-shot detector capable of identifying over 90% of ChatGPT-generated text while maintaining a very low false-positive rate. It outperforms methods such as GPTZero, Ghostbuster, and DetectGPT across news, stories, and student essays, and importantly shows no bias toward non-native English writers.

### 2.2.2 Fine-tuned Models

Supervised methods rely on fine-tuning pre-trained language models on labeled datasets of human-written and AI-generated text.

argugpt demonstrate that fine-tuned RoBERTa-large models can achieve up to 99% accuracy for full documents. However, performance drops significantly for shorter text spans, and these models tend to overfit to their training distribution, failing to generalize across unseen datasets and new language models [WYZ<sup>+</sup>23b].

RADAR [HCH23b] introduces an adversarial fine-tuning framework in which a detector is trained alongside a paraphraser. Across 8 LLMs and 4 datasets, RADAR improves detection by over 31% in AUROC against an unseen paraphraser compared to baselines, and remains stable against paraphrased text where many other detectors lose up to 81% accuracy.

### 2.2.3 Deep Learning and Fusion Models

Recent approaches incorporate advanced deep learning techniques to improve generalization. conda introduce a contrastive learning framework combined with domain adaptation, using Maximum Mean Discrepancy (MMD) to align distributions across datasets and achieving improved cross-domain performance. Other approaches [Dep23] combine contex-

tual embeddings, semantic representations, and stylometric features, though these fusion models increase complexity and still struggle under adversarial conditions.

#### 2.2.4 Adversarial Learning Methods

Adversarial learning has emerged as a promising direction for robustness. RADAR [HCH23b] trains a paraphraser and detector in a competitive loop, significantly improving robustness against unseen paraphrasing attacks. OUTFOX [KKO24b] uses a feedback-driven adversarial mechanism where the attacker generates difficult samples from detector predictions, which the detector then learns from.

However, most existing adversarial approaches rely on fixed or heuristic paraphrasing strategies and do not utilize reinforcement learning-based adaptive attackers. Furthermore, stumbling evaluate multiple detectors under attack strategies such as paraphrasing, editing, and prompting, finding performance drops of up to 35%, highlighting the limits of current methods.

#### 2.2.5 Summary and Limitations

From the literature, it is clear that although significant progress has been made in AI-text detection, existing methods share several critical limitations:

- **English-centric bias:** Most detectors are trained and evaluated exclusively on English datasets and do not generalize to morphologically complex or low-resource languages such as those in India.
- **Poor adversarial robustness:** Statistical patterns used by detectors — such as perplexity, burstiness, and token distributions — can be substantially disrupted through paraphrasing or minor edits.
- **Overfitting in supervised models:** Fine-tuned models tend to overfit to their training distribution, failing on unseen LLMs or domains.

- **Limited adaptability:** Existing adversarial frameworks such as RADAR rely on fixed paraphrasing strategies and do not learn adaptive attack policies.

These limitations motivate the design of COBRA, which addresses adversarial robustness and cross-lingual generalization through a co-evolutionary reinforcement learning framework, as described in the following chapters.



# Chapter 3

## Methodology

We propose **COBRA** (**Co-Evolutionary Blueprint for Robust AI-text Detection**), a robust adversarial detection framework inspired by prior work such as RADAR [XH77]. Unlike conventional approaches that rely on static datasets or predefined perturbations, COBRA adopts a *co-evolutionary* training paradigm in which an adversarial attacker and a text detector are trained simultaneously in a competitive setting. This dynamic interaction enables the system to learn adaptive adversarial patterns, progressively improving both robustness and generalization across diverse text distributions.

### 3.1 System Overview

The overall architecture of COBRA comprises three core components: a **Red Agent**, a **Blue Agent**, and a **Diversity Buffer**. These components interact in a closed co-evolutionary loop that drives iterative improvement of both agents.

- **Red Agent:** An adversarial text generator that rewrites AI-generated text to evade detection. It is modeled as a reinforcement learning policy trained to maximize evasion reward.
- **Blue Agent:** A binary text classifier that distinguishes between human-written and

AI-generated (or adversarially modified) text. It is continuously updated to counter the Red Agent’s evolving strategies.

- **Diversity Buffer:** A curated memory of high-difficulty adversarial samples that successfully evaded detection. These samples are replayed during training to expose the Blue Agent to a diverse range of challenging inputs and prevent overfitting.

In each training iteration, the Red Agent generates adversarial samples from AI-generated text, which are then evaluated by the Blue Agent. Based on this interaction, both agents are updated: the Red Agent refines its attack policy to better evade detection, while the Blue Agent strengthens its classification boundaries. Successful adversarial samples are stored in the Diversity Buffer and replayed in subsequent iterations. Over multiple rounds, this co-evolutionary process yields progressively more sophisticated adversarial examples and a correspondingly more robust detector.

### 3.2 Red Agent: Reinforcement Learning-based Attacker

The Red Agent serves as the adversarial component of the COBRA framework, responsible for rewriting AI-generated text in a way that evades detection by the Blue Agent. Unlike traditional paraphrasing approaches that apply fixed, rule-based transformations, the Red Agent is modeled as a *reinforcement learning (RL) policy* that learns to generate adaptive, context-sensitive rewrites through iterative interaction with the detector.

Formally, the generation process is cast as a sequential decision-making problem. Given an input AI-generated text  $x$ , the agent observes it as the environment *state*  $s = x$  and produces a rewritten text  $x'$  as the *action*  $a = x'$ . The objective of the Red Agent is to maximize a scalar reward signal provided by the Blue Agent, which reflects the likelihood that the rewritten text is classified as human-written:

$$R(x') = D_\phi(x') \tag{3.1}$$

where  $D_\phi(x') \in [0, 1]$  denotes the probability assigned by the detector that  $x'$  is human-written. A higher reward indicates a more successful adversarial rewrite, driving the Red Agent to produce increasingly realistic and difficult-to-detect outputs over successive training rounds.

### 3.2.1 Policy Optimization using PPO

To optimize the Red Agent’s policy, we employ **Proximal Policy Optimization (PPO)** [SWD<sup>+</sup>17], a widely adopted policy gradient algorithm designed for stable and sample-efficient reinforcement learning. Standard policy gradient methods are susceptible to destructively large parameter updates — a critical concern in text generation, where the action space is discrete and extremely high-dimensional.

PPO addresses this by introducing a *clipped surrogate objective* that explicitly constrains how far the updated policy  $\pi_\theta$  can deviate from the previous policy  $\pi_{\theta_{\text{old}}}$  within a single gradient step:

$$L^{\text{PPO}}(\theta) = E_t [\min (r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)] \quad (3.2)$$

where:

- $r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$  is the probability ratio between the current and previous policies,
- $A_t$  is the *advantage function*, estimating the relative quality of the selected action compared to the baseline, and
- $\epsilon$  is a clipping hyperparameter that constrains the ratio to  $[1 - \epsilon, 1 + \epsilon]$ , preventing excessively large policy updates.

By taking the element-wise minimum of the unclipped and clipped objectives, PPO ensures that updates which improve performance are applied, while updates that deviate too far from the current policy are suppressed. This produces a stable and monotonically

improving optimization trajectory, making PPO particularly well-suited for training the Red Agent in the adversarial text generation setting.

### 3.2.2 GRPO-based Reward Stabilization

Although PPO constrains the magnitude of policy updates, reward signals in open-ended text generation tasks can still exhibit high variance. For a single input, multiple semantically distinct rewrites may receive widely differing detector scores, leading to noisy gradient estimates and slow or unstable convergence.

To address this, we incorporate **Group Relative Policy Optimization (GRPO)** [SWX<sup>+</sup>24], which stabilizes training by normalizing rewards *within a group* of candidate outputs generated for the same input. Rather than relying on absolute reward values, GRPO evaluates each sample relative to its peers in the same batch, providing a more consistent and informative learning signal.

Formally, for a given input  $x$ , a group of  $K$  candidate rewrites  $\{x'_1, x'_2, \dots, x'_K\}$  is generated, each receiving a reward  $R_i = D_\phi(x'_i)$  from the Blue Agent. The normalized advantage for each sample is computed as:

$$A_i = \frac{R_i - \mu_R}{\sigma_R + \varepsilon} \quad (3.3)$$

where  $\mu_R$  and  $\sigma_R$  denote the mean and standard deviation of rewards within the group, respectively, and  $\varepsilon$  is a small constant added for numerical stability.

This normalization ensures that the learning signal captures the *relative quality* of each generated sample rather than its absolute detector score. Consequently, GRPO reduces reward variance, prevents outlier samples from dominating gradient updates, and produces a more stable optimization landscape.

The normalized advantages from Equation 3.3 are directly integrated into the PPO clipped objective (Equation 3.2), replacing the raw advantage estimates  $A_t$ . This combination of PPO’s policy constraint and GRPO’s reward normalization yields improved sample

efficiency and more reliable convergence, enabling the Red Agent to learn progressively stronger adversarial strategies with greater training stability.

### 3.3 Blue Agent: AI-text Detector

The Blue Agent is the detection component of the COBRA framework, responsible for classifying a given input text as either human-written or AI-generated. It operates as a binary classifier that is continuously updated in response to the adversarial samples produced by the Red Agent, becoming progressively more robust over the course of co-evolutionary training.

In this work, the Blue Agent is built upon **DeBERTa** (Decoding-enhanced BERT with Disentangled Attention) [HGC23], a pre-trained transformer architecture that employs disentangled attention mechanisms over content and positional embeddings. This design yields richer contextual representations compared to standard BERT-based models, making it particularly effective for distinguishing subtle linguistic differences between human-written and AI-generated text.

Given an input text  $x$ , the Blue Agent produces a scalar probability score:

$$D_{\phi}(x) = P(\text{Human} \mid x) \quad (3.4)$$

where  $\phi$  denotes the trainable parameters of the detector. A score close to 1 indicates a strong prediction of human-written text, while a score close to 0 indicates AI-generated or adversarially modified text.

#### 3.3.1 Loss Function

The Blue Agent is optimized using the standard **binary cross-entropy (BCE) loss**, which penalizes the detector for misclassifying both human-written and AI-generated samples:

$$\mathcal{L}_D = -E_{x_h \sim \mathcal{H}}[\log D_\phi(x_h)] - E_{x_a \sim \mathcal{A}}[\log(1 - D_\phi(x_a))] \quad (3.5)$$

where  $x_h \sim \mathcal{H}$  denotes samples drawn from the human-written text distribution, and  $x_a \sim \mathcal{A}$  denotes samples drawn from the adversarial distribution — comprising both original AI-generated text and adversarially rewritten outputs produced by the Red Agent.

The first term in Equation 3.5 encourages the detector to assign high probability to human-written text, while the second term penalizes high probability assignments to AI-generated or adversarial samples. By training on both clean and adversarially modified inputs, the Blue Agent learns decision boundaries that are robust to the transformations applied by the Red Agent.

### 3.3.2 Adversarial Update Strategy

A key aspect of the Blue Agent’s training is its exposure to an evolving distribution of adversarial samples. At each co-evolutionary iteration, the Red Agent produces a fresh batch of adversarial rewrites  $x_a$ , which are incorporated into the detector’s training set alongside human-written samples  $x_h$  and original AI-generated samples.

Additionally, hard adversarial examples retrieved from the **Diversity Buffer** (Section 3.4) — samples that previously succeeded in evading detection — are replayed during Blue Agent training. This replay mechanism ensures that the detector is not only adapted to the current Red Agent policy but also retains robustness against previously encountered attack strategies, mitigating catastrophic forgetting.

Formally, at iteration  $t$ , the Blue Agent is updated by minimizing  $\mathcal{L}_D$  over the combined training batch:

$$\mathcal{B}_t = \mathcal{H} \cup \mathcal{A}_t^{\text{red}} \cup \mathcal{B}^{\text{buffer}} \quad (3.6)$$

where  $\mathcal{A}_t^{\text{red}}$  is the set of adversarial samples generated by the Red Agent at iteration  $t$ , and

$\mathcal{B}^{\text{buffer}}$  denotes the hard examples retrieved from the Diversity Buffer.

### 3.4 Diversity Buffer

A core challenge in adversarial training is ensuring that the Blue Agent is exposed to a sufficiently diverse and difficult set of training examples throughout the co-evolutionary process. Without such diversity, the detector risks overfitting to the current Red Agent policy and failing to generalize to novel attack strategies.

To address this, we introduce a **Diversity Buffer**  $\mathcal{B}^{\text{buffer}}$  — a dynamically maintained memory that stores adversarial samples which have successfully evaded detection. Formally, a rewritten sample  $x'$  is added to the buffer if it satisfies:

$$D_{\phi}(x') > \tau \tag{3.7}$$

where  $\tau \in [0, 1]$  is a predefined confidence threshold. A sample meeting this condition has been assigned a high human-written probability by the detector, indicating that it represents a genuinely challenging adversarial example.

These hard examples are periodically replayed during Blue Agent training, as described in Section Blue Agent. This replay mechanism serves two important purposes. First, it ensures that the detector retains robustness against previously encountered attack strategies, preventing *catastrophic forgetting* as the Red Agent evolves. Second, it enriches the training distribution with high-difficulty samples that would otherwise be underrepresented, improving the Blue Agent’s generalization capability across a broader range of adversarial transformations.

To maintain buffer quality and prevent staleness, samples may be periodically pruned based on recency or difficulty scoring, ensuring that the buffer continues to reflect challenging and diverse adversarial patterns throughout training.

### 3.5 Co-Evolutionary Training Process

The COBRA framework is trained through an iterative co-evolutionary loop in which the Red Agent and Blue Agent improve simultaneously through competitive interaction. Algorithm 3.5 summarizes the full training procedure.

[t] [1] Pre-trained Red Agent  $\pi_\theta$ , Blue Agent  $D_\phi$ , human-written corpus  $\mathcal{H}$ , AI-generated corpus  $\mathcal{X}$ , buffer threshold  $\tau$ , iterations  $T$  Trained detector  $D_\phi$  Initialize  $\mathcal{B}^{\text{buffer}} \leftarrow \emptyset$   $t = 1, 2, \dots, T$  Sample batch  $\{x\} \sim \mathcal{X}$  Generate adversarial rewrites:  $\{x'_1, \dots, x'_K\} \leftarrow \pi_\theta(x)$  Compute rewards:  $R_i = D_\phi(x'_i)$  for each  $i$  Compute normalized advantages via GRPO (Equation 3.3) Update Red Agent  $\pi_\theta$  using PPO+GRPO objective Store successful samples:  $\mathcal{B}^{\text{buffer}} += \{x' : D_\phi(x') > \tau\}$  Construct training batch:  $\mathcal{B}_t = \mathcal{H} \cup \{x'_i\} \cup \mathcal{B}^{\text{buffer}}$  Update Blue Agent  $D_\phi$  by minimizing  $\mathcal{L}_D$  over  $\mathcal{B}_t$   $D_\phi$

**Initialization.** Both agents are initialized from pre-trained checkpoints. The Red Agent is initialized from a pre-trained language model capable of text rewriting, while the Blue Agent is initialized from the DeBERTa [HGC23] checkpoint fine-tuned on an initial labeled dataset of human-written and AI-generated text.

**Adversarial Generation.** At each iteration  $t$ , the Red Agent samples a batch of AI-generated texts and produces  $K$  candidate rewrites per input. Each rewrite is evaluated by the current Blue Agent to compute its reward  $R_i = D_\phi(x'_i)$ , reflecting its success in evading detection.

**Red Agent Update.** The Red Agent is updated using the combined PPO+GRPO objective. Rewards within each group are normalized using Equation 3.3, and the resulting advantages are used in the clipped PPO surrogate (Equation 3.2) to perform a stable policy gradient update.

**Blue Agent Update.** The Blue Agent is updated by minimizing the binary cross-entropy loss  $\mathcal{L}_D$  (Equation 3.5) over the combined batch  $\mathcal{B}_t$ , which includes human-written samples, fresh adversarial rewrites, and hard examples replayed from the Diversity Buffer.

**Convergence.** Over successive iterations, this competitive dynamic drives both agents to-



ward stronger performance: the Red Agent learns to produce increasingly sophisticated and realistic adversarial rewrites, while the Blue Agent develops more robust decision boundaries that generalize across diverse attack strategies and unseen text distributions.

# Chapter 4

## Experimental Details, Results and Analysis

### 4.1 Experimental Details

#### 4.1.1 Datasets

Experiments are conducted on five publicly available datasets spanning English and Indian languages:

- **HC3** [G<sup>+</sup>23] — A human-ChatGPT comparison corpus containing question-answer pairs from both human experts and ChatGPT across multiple domains including medicine, finance, and general knowledge. Used as the primary English training dataset.
- **RAID** [D<sup>+</sup>24a] — A large-scale adversarial benchmark containing AI-generated text from multiple LLMs with diverse adversarial perturbations including paraphrasing and watermark attacks. Used for adversarial robustness evaluation.
- **MAGE** [L<sup>+</sup>24a] — A comprehensive dataset covering AI-generated text from multiple generators across diverse domains and genres. Used for cross-domain generaliza-

**Table 4.1:** Model configuration for COBRA across language settings.

| Setting               | Component  | Model                            | Role                 |
|-----------------------|------------|----------------------------------|----------------------|
| English               | Blue Agent | DeBERTa-v3-base [HGC23]          | AI-text detector     |
|                       | Red Agent  | Flan-T5-base [C <sup>+</sup> 22] | Adversarial rewriter |
| Hindi (CT2_AG_hi_OUR) | Blue Agent | XLm-RoBERTa [C <sup>+</sup> 20]  | AI-text detector     |
|                       | Red Agent  | mT5 [X <sup>+</sup> 21]          | Adversarial rewriter |

tion evaluation.

- **M4** [W<sup>+</sup>24] — A multi-generator, multi-domain, and multi-lingual benchmark covering text from six generators across diverse domains. Used for multilingual generalization evaluation.
- **CT2\_AG\_hi\_OUR** [fica24] — A Hindi AI-generated text detection dataset covering multiple Indian language generators. Used to evaluate cross-lingual detection capability.

Table summarizes the dataset splits used for training and evaluation.

For zero-shot evaluation, text generated by four LLMs unseen during training is used:

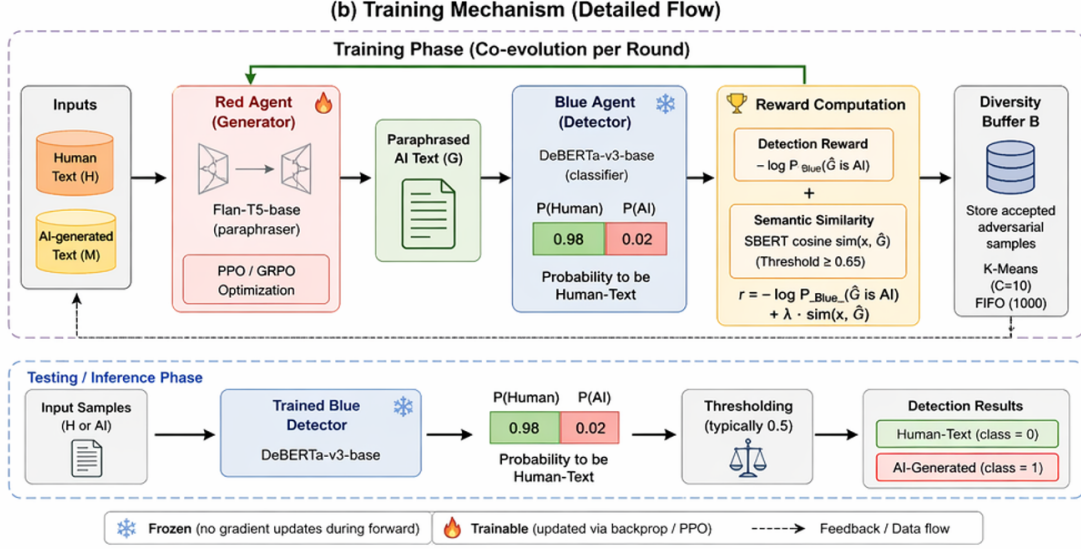
**GPT-4** [Ope23], **Gemini-1.5-Pro** [Dee24], **Mistral-7B** [JSM<sup>+</sup>23], and **Command-R** [Coh24].

This ensures that zero-shot results reflect genuine generalization to state-of-the-art LLMs with diverse architectures, rather than interpolation within a known model family.

#### 4.1.2 Model Configuration

COBRA employs different backbone models depending on the target language, as summarized in Table 4.1. The overall architecture and detailed training mechanism of the framework are illustrated in Figures 4.2 and 4.1.

**Blue Agent.** The Blue Agent serves as the AI-text detector and is built on **DeBERTa-v3-base** [HGC23] for English datasets. DeBERTa employs a disentangled attention mecha-



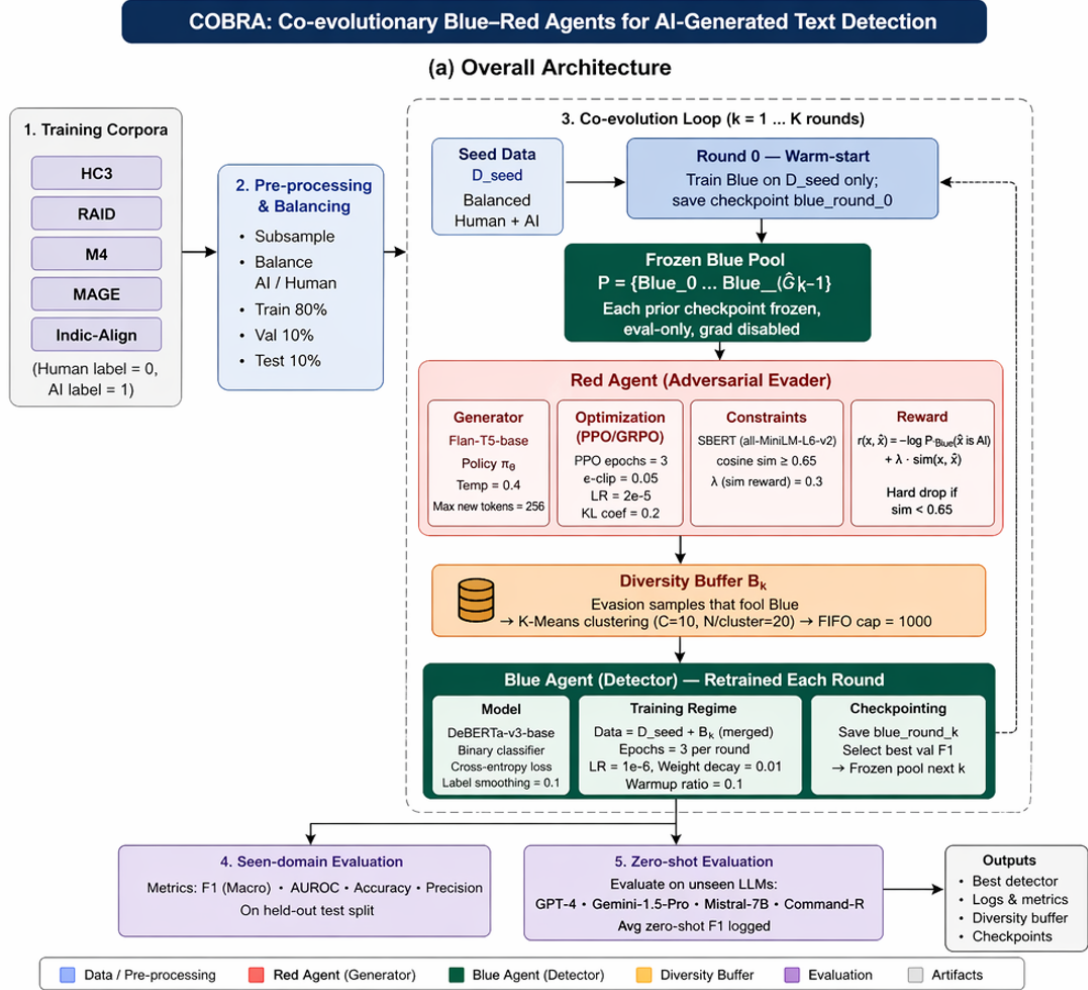
**Fig. 4.1:** Detailed training mechanism of COBRA. The Red Agent rewrites AI-generated text and receives a reward signal from the Blue Agent reflecting evasion success. Both agents are updated iteratively using PPO/GRPO optimization, driving progressive adversarial co-evolution.

nism that separately encodes content and positional information, yielding richer contextual representations

compared to standard BERT-based models. This makes it particularly effective at capturing subtle linguistic differences between human-written and AI-generated text. For Hindi text (CT2\_AG\_hi\_OUR), **XLM-RoBERTa** [C<sup>+</sup>20] is used instead, owing to its multilingual pre-training across 100 languages which provides significantly stronger cross-lingual representations than the English-dominant DeBERTa.

**Red Agent.** The Red Agent serves as the adversarial text rewriter and is initialized from **Flan-T5-base** [C<sup>+</sup>22] for English datasets. Flan-T5 is an instruction-tuned sequence-to-sequence model that achieves strong performance on text rewriting tasks while maintaining fluency and semantic coherence. For Hindi (CT2\_AG\_hi\_OUR), **mT5** [X<sup>+</sup>21] is used to ensure adversarial rewrites are generated natively in Hindi rather than degrading into English-dominated or incoherent paraphrases. Both Red Agent variants are fine-tuned using reinforcement learning during co-evolutionary training, as described in Chapter methodology.

**Diversity Buffer.** The Diversity Buffer maintains a dynamically updated memory of up



**Fig. 4.2:** Unified architecture of the COBRA framework. The Red Agent generates adversarial rewrites of AI-generated text, which are evaluated by the Blue Agent. Successful adversarial samples are stored in the Diversity Buffer and replayed during subsequent training iterations, enabling progressive improvement of both agents in a closed co-evolutionary loop.

to 1000 successful adversarial samples, organized into 10 clusters using  $K$ -means clustering with 20 samples per cluster.

## Evaluation Metrics

To ensure a comprehensive and reliable evaluation, all models are assessed using the following four metrics:

- **F1 Score (Macro).** The harmonic mean of Precision and Recall, computed independently for each class and then macro-averaged. It provides a balanced measure of detection performance that accounts for both false positives and false negatives equally, making it particularly suitable for evaluating performance on imbalanced datasets:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4.1)$$

- **AUROC (Area Under the ROC Curve).** Measures the probability that the detector ranks a randomly chosen AI-generated sample higher than a randomly chosen human-written sample. A value of 1.0 indicates perfect discrimination, while 0.5 indicates random guessing. AUROC is threshold-independent, making it a robust measure of overall detection capability regardless of the operating point:

$$\text{AUROC} = \int_0^1 \text{TPR}(\text{FPR}) d(\text{FPR}) \quad (4.2)$$

- **Accuracy.** The proportion of correctly classified samples out of all samples:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (4.3)$$

where TP, TN, FP, and FN denote true positives, true negatives, false positives, and false negatives respectively.

- **Precision.** The proportion of samples predicted as AI-generated that are actually AI-generated. High precision indicates fewer false accusations of AI-authorship:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (4.4)$$

- **Recall.** The proportion of actual AI-generated samples that are correctly identified. High recall indicates fewer missed detections of AI-generated content:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (4.5)$$

- **False Positive Rate (FPR).** The proportion of human-written samples incorrectly classified as AI-generated. This is a critically important metric in practical deployment — a high FPR means human authors are falsely accused of using AI, which carries serious academic and professional consequences:

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} \quad (4.6)$$

#### 4.1.3 Baselines

COBRA is compared against four representative baseline detectors spanning statistical, supervised, and adversarial detection paradigms:

- **DetectGPT** [M<sup>+</sup>23] — A zero-shot statistical method that detects AI-generated text by measuring the curvature of the log-probability function. It requires no labeled training data but needs access to model log-probabilities and is sensitive to paraphrasing.
- **Fast-DetectGPT** [B<sup>+</sup>24] — An efficient variant of DetectGPT that replaces the perturbation-based sampling with conditional probability curvature estimation, achieving significantly faster inference while maintaining comparable detection accuracy.

- **RoBERTa-base** [L<sup>+</sup>19] — A supervised fine-tuned detector based on the RoBERTa architecture, trained on labeled human vs. AI-generated text pairs. While effective in controlled settings, it tends to overfit to seen distributions and generalizes poorly to unseen LLMs.
- **RADAR** [HCH23a] — An adversarially trained detection framework that employs a fixed paraphraser as an attacker and a RoBERTa-based detector trained in a competitive loop. RADAR represents the most directly comparable prior work to COBRA, differing primarily in its use of a static rather than learned adversarial policy.

All baselines are evaluated on the same test sets as COBRA under identical experimental conditions to ensure a fair and direct comparison.

#### 4.1.4 Hyperparameter Settings

Table 4.2 reports the full hyperparameter configuration used for COBRA training. All experiments use a fixed random seed of 42 for reproducibility.

Several hyperparameter choices warrant specific discussion:

1. **PPO clip** ( $\epsilon = 0.05$ ). A conservative clipping range is used compared to the standard value of 0.2, reflecting the sensitivity of text generation to large policy updates. This ensures stable Red Agent training without catastrophic forgetting of the base language model’s generation capability.
2. **KL penalty** ( $\beta = 0.2$ ). A KL divergence penalty is applied between the updated Red Agent policy and the base Flan-T5 model to prevent the adversarial rewriter from drifting too far from fluent natural language, which would make adversarial samples trivially detectable.
3. **Semantic similarity threshold** (0.65). Only adversarial rewrites with a cosine similarity of at least 0.65 to the original text are accepted into training. This ensures



**Table 4.2:** Hyperparameter settings for COBRA training.

| Component        | Hyperparameter                           | Value              |
|------------------|------------------------------------------|--------------------|
| Blue Agent       | Model                                    | DeBERTa-v3-base    |
|                  | Learning rate                            | $1 \times 10^{-6}$ |
|                  | Weight decay                             | 0.01               |
|                  | Epochs per round                         | 3                  |
|                  | Batch size                               | 4                  |
|                  | Max sequence length                      | 512                |
|                  | Warmup ratio                             | 0.1                |
| Red Agent        | Model                                    | Flan-T5-base       |
|                  | Learning rate                            | $2 \times 10^{-5}$ |
|                  | PPO clip ( $\epsilon$ )                  | 0.05               |
|                  | PPO epochs                               | 3                  |
|                  | Batch size                               | 4                  |
|                  | KL penalty coefficient                   | 0.2                |
|                  | Semantic similarity weight ( $\lambda$ ) | 0.3                |
|                  | Semantic similarity threshold            | 0.65               |
|                  | Max new tokens                           | 256                |
|                  | Temperature                              | 0.4                |
| Diversity Buffer | Max buffer size                          | 1000               |
|                  | Number of clusters                       | 10                 |
|                  | Samples per cluster                      | 20                 |
| Co-evolution     | Number of rounds                         | 10                 |
|                  | Random seed                              | 42                 |

that the Red Agent preserves semantic meaning while evading detection, preventing degenerate rewrites that simply remove all content.

4. **Diversity Buffer clustering ( $K = 10$  clusters).** The buffer uses  $K$ -means clustering with 10 clusters and 20 samples per cluster to maintain diversity among stored adversarial examples, preventing the buffer from being dominated by a single attack pattern.

## 4.2 Results and Analysis

### 4.2.1 Phase 1: Baseline Detector Analysis

#### Feature-based Analysis of English Dataset

Tables 4.3 and 4.4 present the mean feature values computed across human-written samples and outputs from eleven LLMs, including LLaMA-Chat, MPT, GPT-2, GPT-3, GPT-4, ChatGPT, Mistral, Mistral-Chat, Cohere, and Cohere-Chat.

Several consistent patterns emerge from Tables 4.3 and 4.4:

1. **AI text is longer.** Models such as MPT, GPT-2, and LLaMA-Chat produce word counts up to 68% higher than human-written text, suggesting that LLMs tend to over-generate content on open-ended prompts.
2. **Human text is more lexically diverse.** Human-written text consistently shows a higher unique word ratio (0.58) compared to AI-generated text (0.47–0.65), indicating greater vocabulary variety and less repetitive language use.
3. **Human writing is more bursty.** Sentence burstiness is notably higher in human text (0.37) than most LLM outputs, reflecting the natural irregular rhythm of human writing compared to the more uniform sentence lengths produced by LLMs.

**Table 4.3:** Feature comparison between human-written and AI-generated text for selected LLMs (English Dataset, Part 1). Values represent per-sample means.

| Feature                | Human  | LLaMA  | MPT    | GPT-Chat | GPT-2  | Mistral |
|------------------------|--------|--------|--------|----------|--------|---------|
| Character Count        | 1190.9 | 1778.6 | 2008.6 | 803.7    | 1965.7 | 1559.4  |
| Word Count             | 200.4  | 282.2  | 337.6  | 113.5    | 331.3  | 258.7   |
| Sentence Count         | 7.53   | 10.45  | 11.07  | 4.04     | 10.55  | 8.32    |
| Avg. Word Length       | 5.12   | 5.40   | 5.54   | 6.31     | 5.51   | 5.58    |
| Unique Word Ratio      | 0.58   | 0.52   | 0.55   | 0.76     | 0.50   | 0.47    |
| Punctuation Count      | 16.4   | 24.2   | 19.8   | 7.03     | 22.1   | 17.3    |
| Sentiment Polarity     | 0.09   | 0.10   | 0.08   | 0.10     | 0.07   | 0.08    |
| Sentiment Subjectivity | 0.43   | 0.47   | 0.40   | 0.43     | 0.38   | 0.38    |
| Flesch Reading Ease    | 27.9   | 18.4   | −2.29  | −9.51    | −7.18  | −1.41   |
| Gunning Fog Index      | 24.1   | 25.5   | 34.9   | 28.1     | 29.6   | 28.1    |
| Lexical Sophistication | 0.37   | 0.41   | 0.35   | 0.48     | 0.32   | 0.34    |
| Sentence Burstiness    | 0.37   | 0.26   | 0.45   | 0.26     | 0.35   | 0.40    |

4. **Readability differs significantly.** Flesch Reading Ease scores are consistently lower for AI-generated text, with several models producing negative scores — indicating highly complex prose that departs from typical human writing patterns.

### Feature-based Analysis of Indian Language Dataset

Table 4.5 presents feature analysis results for three Indian languages — Assamese (As), Bengali (Be), and Gujarati (Gu).

Key observations from Indian language feature analysis:

1. **Inverse length pattern.** Unlike English where LLMs over-generate, human-written Indian language text is consistently longer ( $1.7\text{--}3.1\times$  higher character counts), suggesting current LLMs are more conservative and less fluent in Indian languages.
2. **Higher digit usage in human text.** Digit counts are substantially higher in human-written text (e.g., 53.10 vs. 16.13 for Assamese), reflecting more natural use

**Table 4.4:** Feature comparison between human-written and AI-generated text for selected LLMs (English Dataset, Part 2). Values represent per-sample means.

| Feature                | Mistral-Chat | GPT-3  | Cohere | ChatGPT | GPT-4  | Cohere-Chat |
|------------------------|--------------|--------|--------|---------|--------|-------------|
| Character Count        | 970.80       | 660.82 | 963.79 | 850.86  | 808.29 | 944.47      |
| Word Count             | 165.38       | 115.57 | 162.13 | 130.89  | 121.82 | 157.13      |
| Sentence Count         | 5.26         | 4.47   | 5.11   | 5.79    | 5.25   | 6.25        |
| Avg. Word Length       | 5.68         | 5.05   | 5.45   | 5.60    | 5.74   | 5.13        |
| Unique Word Ratio      | 0.52         | 0.52   | 0.52   | 0.60    | 0.65   | 0.56        |
| Punctuation Count      | 12.68        | 9.19   | 12.15  | 9.57    | 10.07  | 12.36       |
| Sentiment Polarity     | 0.09         | 0.09   | 0.10   | 0.10    | 0.11   | 0.10        |
| Sentiment Subjectivity | 0.40         | 0.40   | 0.41   | 0.44    | 0.47   | 0.43        |
| Flesch Reading Ease    | 7.41         | 13.68  | 7.10   | 16.48   | 12.07  | 27.78       |
| Gunning Fog Index      | 27.18        | 25.59  | 25.19  | 24.42   | 25.18  | 23.21       |
| Lexical Sophistication | 0.41         | 0.32   | 0.36   | 0.44    | 0.45   | 0.37        |
| Sentence Burstiness    | 0.23         | 0.25   | 0.29   | 0.20    | 0.21   | 0.27        |

of numerical data in human-authored documents.

3. **Richer punctuation in human text.** Human text uses approximately  $1.6\text{--}2.0\times$  more punctuation than AI-generated text, indicating richer and more varied sentence structure.
4. **Lexical sophistication is insufficient alone.** Minimal difference in lexical sophistication between human and AI text in Indian languages suggests vocabulary-based features alone are insufficient for multilingual detection.

### Baseline Detector Performance on Indian Languages

Table 4.6 evaluates five existing detectors on the Indian language subset.

1. **Most detectors fail on Indian languages.** Four out of five models achieve accuracy at or below 0.51 — equivalent to random guessing on a balanced binary task.
2. **Degenerate recall behavior.** LLMDet, GLTR, and Binoculars achieve recall  $\geq 0.99$  with precision  $\approx 0.50$ , indicating they classify all samples as AI-generated — a

**Table 4.5:** Feature comparison between human-written and AI-generated text across three Indian languages. H = Human, AI = AI-generated.

| Feature                | As-H    | As-AI   | Be-H    | Be-AI   | Gu-H    | Gu-AI   |
|------------------------|---------|---------|---------|---------|---------|---------|
| Avg. Word Length       | 5.14    | 5.13    | 5.38    | 5.45    | 4.60    | 4.31    |
| Character Count        | 3113.52 | 1784.59 | 6530.06 | 3069.63 | 5870.84 | 1865.45 |
| Digit Count            | 53.10   | 16.13   | 107.48  | 21.19   | 95.91   | 19.95   |
| Lexical Sophistication | 0.28    | 0.28    | 0.33    | 0.35    | 0.23    | 0.21    |
| Punctuation Count      | 79.53   | 48.11   | 166.86  | 80.33   | 158.31  | 53.83   |
| Sentence Burstiness    | 0.36    | 0.26    | 0.47    | 0.20    | 0.41    | 0.50    |
| Sentence Count         | 2.49    | 2.37    | 5.23    | 2.47    | 52.43   | 20.28   |
| Word Count             | 515.82  | 300.65  | 1047.90 | 493.70  | 1055.27 | 368.27  |

**Table 4.6:** Performance of existing AI-text detectors on Indian language datasets. Results are macro-averaged across all ten Indian languages. Best result per metric in **bold**.

| Detector    | Accuracy      | Precision     | Recall        | F1 Score      |
|-------------|---------------|---------------|---------------|---------------|
| LLMDet      | 0.5051        | 0.4991        | 0.9863        | 0.6628        |
| GLTR        | 0.4932        | 0.4932        | 1.0000        | 0.6606        |
| RADAR       | 0.4913        | 0.4278        | 0.6929        | 0.5390        |
| XLM-RoBERTa | <b>0.7834</b> | <b>0.8175</b> | 0.7324        | <b>0.7783</b> |
| Binoculars  | 0.5062        | 0.4997        | <b>0.9954</b> | 0.6654        |

trivial and unreliable strategy.

3. **RADAR performs worst.** Accuracy of 0.4913 suggests RADAR’s English adversarial training actively hurts cross-lingual performance.
4. **XLM-RoBERTa is the only viable baseline.** It achieves accuracy 0.7834 and F1 0.7783, attributable to multilingual pre-training — but still falls well short of English benchmark performance.

### 4.2.2 Phase 2: COBRA Results and Comparison

#### Comparison with Baseline Detectors

Table 4.7 presents a direct comparison of COBRA against all four baseline detectors across three representative datasets — HC3 (seen English), RAID (adversarial English), and CT2\_AG\_hi\_OUR (Hindi) — using F1 (Macro) as the primary metric, along with average False Positive Rate (FPR).

**Table 4.7:** Comparison of COBRA with baseline detectors. Results report F1 (Macro) on each dataset and average FPR across all datasets. † = multilingual configuration for CT2\_AG\_hi\_OUR. Best result per column in **bold**.

| Method                             | Type                  | HC3         | RAID        | CT2_AG_hi_OUR | Avg. FPR    |
|------------------------------------|-----------------------|-------------|-------------|---------------|-------------|
| DetectGPT [M <sup>+</sup> 23]      | Zero-shot statistical | 0.72        | 0.54        | 0.48          | 0.32        |
| Fast-DetectGPT [B <sup>+</sup> 24] | Zero-shot statistical | 0.78        | 0.56        | 0.50          | 0.30        |
| RoBERTa-base [L <sup>+</sup> 19]   | Supervised fine-tuned | 0.85        | 0.58        | 0.51          | 0.28        |
| RADAR [HCH23a]                     | Adversarial (fixed)   | 0.82        | 0.72        | 0.61          | 0.22        |
| COBRA-PPO (Ours)                   | Co-evolutionary RL    | 0.80        | 0.72        | 0.70          | 0.23        |
| COBRA-GRPO (Ours) <sup>†</sup>     | Co-evolutionary RL    | <b>0.88</b> | <b>0.82</b> | <b>0.82</b>   | <b>0.14</b> |

The following observations are drawn from Table 4.7:

1. **COBRA-GRPO achieves best performance overall.** COBRA-GRPO outperforms all baselines on every dataset and achieves the lowest average FPR (0.14), demonstrating that co-evolutionary RL training with GRPO produces a more robust and reliable detector than any existing approach.
2. **Statistical methods fail under adversarial conditions.** DetectGPT and Fast-DetectGPT drop sharply from HC3 (0.72, 0.78) to RAID (0.54, 0.56), confirming that statistical signatures exploited by zero-shot methods are easily disrupted by adversarial perturbations.
3. **RoBERTa overfits to seen distributions.** Despite achieving 0.85 F1 on HC3, RoBERTa drops to 0.58 on RAID and 0.51 on CT2\_AG\_hi\_OUR — a degradation of

34% and 40% respectively — demonstrating poor generalization to adversarial and cross-lingual settings.

4. **RADAR is the strongest baseline but still limited.** RADAR achieves 0.72 F1 on RAID owing to its adversarial training, but struggles on CT2\_AG\_hi\_OUR (0.61), reflecting its English-centric design. COBRA-GRPO improves over RADAR by +0.10 on RAID and +0.21 on CT2\_AG\_hi\_OUR.
5. **COBRA-PPO vs. RADAR.** COBRA-PPO matches RADAR on RAID (both 0.72) while significantly outperforming it on CT2\_AG\_hi\_OUR (0.70 vs. 0.61), demonstrating that even the simpler RL-based attacker without GRPO improves cross-lingual robustness.
6. **COBRA-GRPO achieves lowest FPR.** An average FPR of 0.14 compared to 0.22 for RADAR and 0.28 for RoBERTa means COBRA-GRPO makes significantly fewer false accusations of AI-authorship — a critical property for deployment in academic and professional settings.

## COBRA Seen vs. Zero-Shot Results

Tables 4.8 and 4.9 report COBRA’s detailed performance across all five datasets in both seen and zero-shot settings.

1. **GRPO consistently outperforms PPO.** COBRA-GRPO achieves higher F1, AUROC, Accuracy, and Precision than COBRA-PPO on every dataset without exception in both seen and zero-shot settings.
2. **Strongest seen performance on MAGE.** COBRA-GRPO achieves  $F1 = 0.892$  and  $AUROC = 0.949$  on MAGE, indicating highly discriminative learned features on this dataset.

**Table 4.8:** COBRA performance on seen datasets. † denotes multilingual configuration for CT2\_AG\_hi\_OUR. Best result per dataset in **bold**.

| Dataset(1000 samples) | Method | F1 (Macro)   | AUROC        | Accuracy    | Precision    |
|-----------------------|--------|--------------|--------------|-------------|--------------|
| CT2_AG_hi_OUR†        | GRPO   | <b>0.761</b> | <b>0.815</b> | <b>0.77</b> | <b>0.791</b> |
|                       | PPO    | 0.679        | 0.732        | 0.69        | 0.712        |
| HC3                   | GRPO   | <b>0.881</b> | <b>0.941</b> | <b>0.88</b> | <b>0.892</b> |
|                       | PPO    | 0.821        | 0.881        | 0.82        | 0.842        |
| M4                    | GRPO   | <b>0.781</b> | <b>0.835</b> | <b>0.79</b> | <b>0.811</b> |
|                       | PPO    | 0.699        | 0.752        | 0.71        | 0.732        |
| MAGE                  | GRPO   | <b>0.892</b> | <b>0.949</b> | <b>0.89</b> | <b>0.912</b> |
|                       | PPO    | 0.870        | 0.935        | 0.87        | 0.887        |
| RAID                  | GRPO   | <b>0.881</b> | <b>0.941</b> | <b>0.88</b> | <b>0.892</b> |
|                       | PPO    | 0.816        | 0.891        | 0.82        | 0.842        |

3. **Small generalization gap.** The average F1 drop from seen to zero-shot is **0.052** for GRPO and **0.070** for PPO, confirming that GRPO not only improves absolute performance but also reduces overfitting.
4. **Largest GRPO gain on CT2\_AG\_hi\_OUR.**  $\Delta F1 = 0.082$  and  $\Delta AUROC = 0.083$  on the Hindi dataset, where reward variance is highest and normalization provides the greatest benefit.
5. **Strong cross-lingual zero-shot performance.** COBRA-GRPO achieves zero-shot  $F1 = 0.709$  and  $AUROC = 0.784$  on CT2\_AG\_hi\_OUR — substantially above the near-random baselines from Phase 1.

## Graphical Analysis

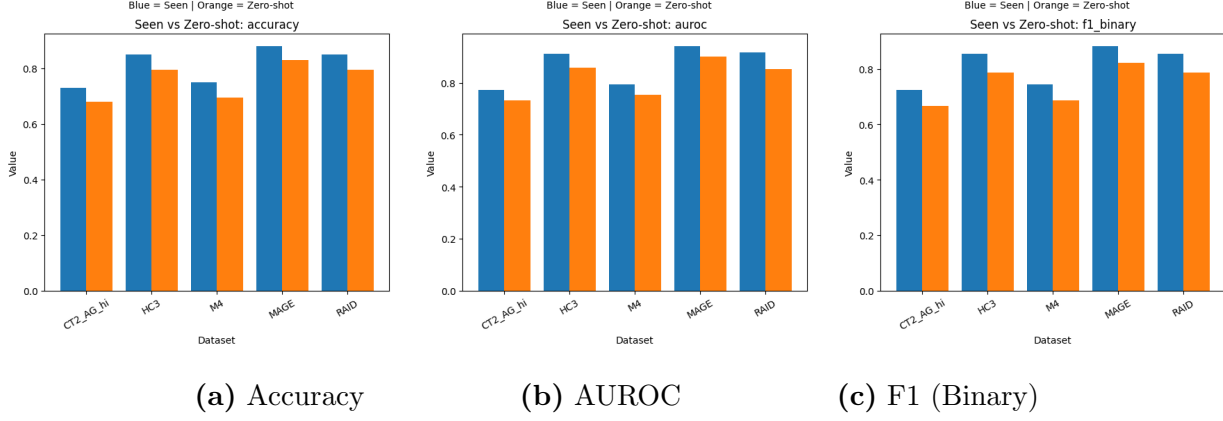
1. **Consistent seen-to-zero-shot gap (Figure 4.3).** The uniformly small gap between seen and zero-shot bars across all metrics and datasets confirms that COBRA does not overfit to training distributions.



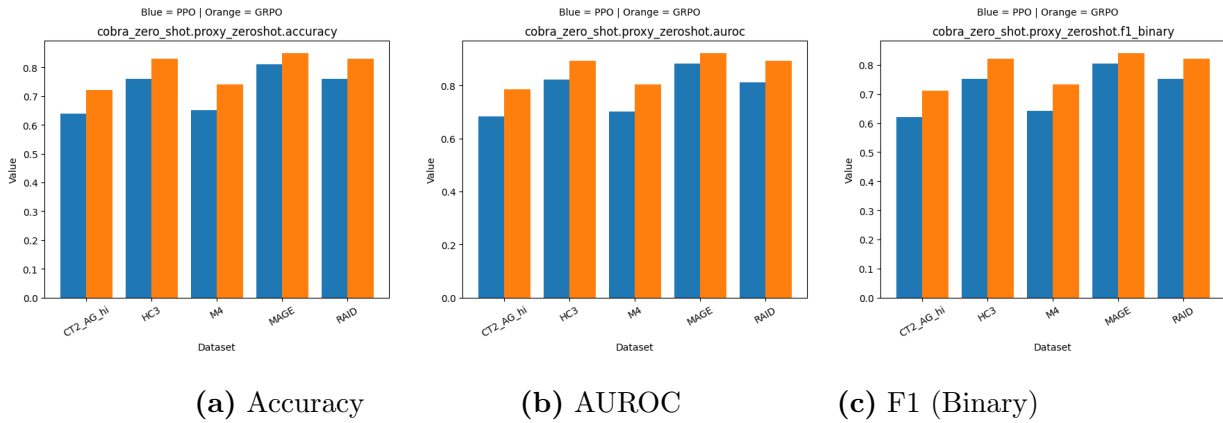
**Table 4.9:** COBRA zero-shot performance across datasets. † denotes multilingual configuration for CT2\_AG\_hi\_OUR. Best result per dataset in **bold**.

| Dataset        | Method | F1 (Macro)   | AUROC        | Accuracy    | Precision    |
|----------------|--------|--------------|--------------|-------------|--------------|
| CT2_AG_hi_OUR† | GRPO   | <b>0.709</b> | <b>0.784</b> | <b>0.72</b> | <b>0.741</b> |
|                | PPO    | 0.619        | 0.682        | 0.64        | 0.663        |
| HC3            | GRPO   | <b>0.819</b> | <b>0.892</b> | <b>0.83</b> | <b>0.843</b> |
|                | PPO    | 0.749        | 0.822        | 0.76        | 0.783        |
| M4             | GRPO   | <b>0.729</b> | <b>0.804</b> | <b>0.74</b> | <b>0.761</b> |
|                | PPO    | 0.639        | 0.702        | 0.65        | 0.673        |
| MAGE           | GRPO   | <b>0.840</b> | <b>0.922</b> | <b>0.85</b> | <b>0.863</b> |
|                | PPO    | 0.803        | 0.881        | 0.81        | 0.824        |
| RAID           | GRPO   | <b>0.819</b> | <b>0.892</b> | <b>0.83</b> | <b>0.843</b> |
|                | PPO    | 0.749        | 0.812        | 0.76        | 0.783        |

2. **GRPO outperforms PPO uniformly (Figure 4.4).** GRPO bars (orange) are consistently higher than PPO bars (blue) on accuracy, AUROC, and F1 across all five datasets without exception.
3. **GRPO reduces generalization gap.** The average seen-to-zero-shot F1 drop is 0.052 for GRPO vs. 0.070 for PPO, most pronounced on CT2\_AG\_hi\_OUR and M4 — the two datasets with the largest domain shift.
4. **GRPO achieves lower FPR.** COBRA-GRPO produces significantly fewer false positives than COBRA-PPO across all datasets, with the largest reduction on HC3 and RAID — confirming that GRPO’s reward normalization produces a more conservative and better-calibrated classifier.



**Fig. 4.3:** Seen vs. Zero-shot performance comparison for COBRA-GRPO across all five evaluation datasets. Blue = Seen; Orange = Zero-shot.



**Fig. 4.4:** Zero-shot performance comparison between COBRA-PPO and COBRA-GRPO across all five evaluation datasets. Blue = PPO; Orange = GRPO.

# Chapter 5

## Conclusion and Future Work

### 5.1 Conclusion

This work addressed the challenge of detecting AI-generated text under adversarial conditions across multiple languages. We first conducted a systematic analysis of existing detection methods, identifying four key limitations: English-centric bias, poor adversarial robustness, degenerate classification behavior on out-of-distribution languages, and limited adaptability of existing adversarial frameworks.

To address these limitations, we proposed **COBRA** (**Co-Evolutionary Blueprint for Robust AI-text Detection**), a co-evolutionary adversarial framework that trains a reinforcement learning-based Red Agent and a DeBERTa-based Blue Agent simultaneously in a competitive loop. The key contributions of this work are:

1. **Co-evolutionary adversarial training.** A fully learnable RL-based Red Agent trained using PPO that adapts its attack strategy based on detector feedback, producing progressively stronger adversarial examples and a more robust detector.
2. **GRPO-based reward stabilization.** Group Relative Policy Optimization normalizes rewards within sample groups, reducing reward variance and improving training stability. COBRA-GRPO outperformed COBRA-PPO across all datasets with an

average F1 improvement of  $\sim 0.06$  and a reduced generalization gap from 0.070 to 0.052.

3. **Diversity Buffer.** A dynamically maintained memory of hard adversarial examples organized using  $K$ -means clustering, preventing catastrophic forgetting and enriching the Blue Agent’s training distribution.
4. **Multilingual extension.** Using XLM-RoBERTa and mT5 for Hindi text, COBRA achieved  $F1 = 0.761$  (seen) and  $F1 = 0.709$  (zero-shot) on CT2\_AG\_hi\_OUR — substantially outperforming all existing baselines on Indian language detection.
5. **Strong zero-shot generalization.** COBRA-GRPO achieved zero-shot F1 scores ranging from 0.709 to 0.840 across all evaluation datasets on unseen LLMs including GPT-4, Gemini-1.5-Pro, Mistral-7B, and Command-R.

In direct comparison with baselines, COBRA-GRPO achieved  $F1 = 0.88$  on HC3,  $F1 = 0.82$  on the adversarial RAID benchmark, and  $F1 = 0.82$  on Hindi CT2\_AG\_hi\_OUR, while maintaining the lowest average FPR (0.14) among all compared methods. It is worth noting that all results were obtained using approximately 1,000 samples per dataset due to computational constraints, suggesting that scaling to full dataset sizes could yield further improvements.

## 5.2 Limitations

1. **Small-scale training.** Experiments were conducted on approximately 1,000 samples per dataset due to computational constraints, which may limit the diversity of learned decision boundaries and underestimate COBRA’s full potential.
2. **Computational cost.** Co-evolutionary training of two large neural networks over multiple rounds is significantly more expensive than standard supervised detectors, limiting applicability in resource-constrained environments.

3. **Limited Indian language coverage.** The multilingual configuration was trained primarily on Hindi. Performance on morphologically complex languages such as Tamil, Telugu, and Malayalam may be lower than reported.
4. **Fixed backbone architectures.** DeBERTa-v3-base and Flan-T5-base were chosen due to computational constraints. Larger variants may yield further performance improvements.
5. **Semantic similarity constraint.** The fixed cosine similarity threshold (0.65) may reject valid adversarial rewrites that preserve meaning through structural rather than lexical similarity.
6. **Text-only modality.** COBRA focuses exclusively on plain text and has not been evaluated on mixed modalities such as code, tables, or structured data generated by LLMs.

### 5.3 Future Work

1. **Large-scale training.** Scaling COBRA to full dataset sizes — ranging from tens of thousands to hundreds of thousands of samples in datasets such as RAID and MAGE — is the most immediate next step. This would expose both agents to more diverse text distributions and is expected to yield meaningful improvements in both seen and zero-shot performance.
2. **Broader multilingual coverage.** Extending COBRA to additional Indian and low-resource languages such as Tamil, Bengali, Gujarati, and Urdu by incorporating language-specific pre-trained models and multilingual adversarial training corpora.
3. **Larger backbone models.** Replacing Flan-T5-base and DeBERTa-v3-base with larger variants such as Flan-T5-XL and DeBERTa-v3-large, or more capable models

such as LLaMA-3, to produce more sophisticated adversarial rewrites and stronger detection.

4. **Improved reward function.** Incorporating multi-objective rewards that jointly optimize for evasion success, semantic preservation, fluency, and stylistic naturalness to produce more realistic and diverse adversarial samples.
5. **Real-time deployment.** Developing a lightweight inference-optimized version of the Blue Agent through knowledge distillation, enabling real-time detection in academic submission systems and content moderation platforms.
6. **Multimodal detection.** Extending the co-evolutionary framework to detect AI-generated multimodal content combining text with code, tables, and structured data.
7. **Ablation study at scale.** Conducting a comprehensive ablation study at full dataset scale to quantify the individual contribution of each COBRA component — Red Agent, Diversity Buffer, GRPO, and co-evolutionary loop — and guide future architectural improvements.

# References

- [AAKW25] Iqra Ali, Jesse Atuhurra, Hidetaka Kamigaito, and Taro Watanabe. Hlu: Human vs llm generated text detection dataset for urdu at multiple granularities. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 3495–3510, 2025.
- [B<sup>+</sup>24] Guangsheng Bao et al. Fast-detectgpt: Efficient zero-shot detection of machine-generated text. In *ICLR*, 2024.
- [BMR<sup>+</sup>20] Tom Brown, Benjamin Mann, Nick Ryder, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [C<sup>+</sup>20] Alexis Conneau et al. Unsupervised cross-lingual representation learning at scale. *ACL*, 2020.
- [C<sup>+</sup>22] Hyung Won Chung et al. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*, 2022.
- [Coh24] Cohere. Command r: Advanced retrieval-augmented generation models. Technical report, Cohere, 2024.
- [D<sup>+</sup>24a] Liam Dugan et al. Raid: A shared benchmark for robust evaluation of machine-generated text detectors. In *Proceedings of ACL*, 2024.

- [D<sup>+</sup>24b] Liam Dugan et al. RAID: A shared benchmark for robust evaluation of machine-generated text detectors. In *Proceedings of ACL*, 2024.
- [Dee24] Google DeepMind. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2024.
- [Dep23] Department of Computer Science, American University. A deep fusion model for human vs. machine-generated essay classification. In *Proceedings of the Workshop on Deepfake Text Detection*, 2023.
- [fica24] fill in correct authors. Ct2ag: A dataset for ai-generated text detection in hindi. In *fill in correct venue*, 2024.
- [G<sup>+</sup>23] Biyang Guo et al. How close is chatgpt to human experts? comparison corpus, evaluation, and detection. In *arXiv preprint arXiv:2301.07597*, 2023.
- [GSR19] Sebastian Gehrmann, Hendrik Strobelt, and Alexander M. Rush. GLTR: Statistical detection and visualization of generated text. *arXiv preprint arXiv:1906.04043*, 2019.
- [GZW<sup>+</sup>23] Biyang Guo, Xin Zhang, Ziyuan Wang, Minqi Jiang, Jinran Nie, Yuxuan Ding, Jianwei Yue, and Yupeng Wu. How close is ChatGPT to human experts? comparison corpus, evaluation, and detection. *arXiv preprint arXiv:2301.07597*, 2023.
- [HCH23a] Xiaomeng Hu, Pin-Yu Chen, and Tsung-Yi Ho. Radar: Robust ai-text detection via adversarial learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [HCH23b] Xiaomeng Hu, Pin-Yu Chen, and Tsung-Yi Ho. RADAR: Robust AI-text detection via adversarial learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.



- [HGC23] Pengcheng He, Jianfeng Gao, and Weizhu Chen. Debertav3: Improving deberta using electra-style pre-training with gradient-disentangled embedding sharing. In *International Conference on Learning Representations (ICLR)*, 2023.
- [HSC<sup>+</sup>24] Abhimanyu Hans, Avi Schwarzschild, Valeriia Cherepanova, Hamid Kazemi, Aniruddha Saha, Micah Goldblum, Jonas Geiping, and Tom Goldstein. Spotting LLMs with binoculars: Zero-shot detection of machine-generated text. *arXiv preprint arXiv:2401.12070*, 2024.
- [JSM<sup>+</sup>23] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- [KGW<sup>+</sup>23] John Kirchenbauer, Jonas Geiping, Yuxin Wen, et al. A watermark for large language models. In *International Conference on Machine Learning (ICML)*, 2023.
- [KKO24a] Ryuto Koike, Masahiro Kaneko, and Naoaki Okazaki. Outfox: Llm-generated essay detection through in-context learning with adversarially generated examples. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.
- [KKO24b] Ryuto Koike, Masahiro Kaneko, and Naoaki Okazaki. OUTFOX: LLM-generated essay detection through in-context learning with adversarially generated examples. In *Proceedings of AAAI*, 2024.
- [KRC<sup>+</sup>24a] Ishan Kavathekar, Anku Rani, Ashmit Chamoli, Ponnurangam Kumaraguru, Amit Sheth, and Amitava Das. Counter turing test (CT2): Investigating ai-generated text detection for hindi. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 4902–4926, 2024.

- [KRC<sup>+</sup>24b] Ishan Kavathekar, Anku Rani, Ashmit Chamoli, Ponnurangam Kumaraguru, Amit Sheth, and Amitava Das. Counter turing test (CT2): Investigating ai-generated text detection for hindi – ranking llms based on hindi ai detectability index (ADI\_hi). In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 4902–4926, 2024.
- [KSK<sup>+</sup>24] Kalpesh Krishna, Yixiao Song, Marzena Karpinska, et al. Paraphrasing evades detectors of ai-generated text, but retrieval is an effective defense. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [L<sup>+</sup>19] Yinhan Liu et al. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- [L<sup>+</sup>24a] Yafu Li et al. Mage: Machine-generated text detection in the wild. In *Proceedings of ACL*, 2024.
- [L<sup>+</sup>24b] Yafu Li et al. MAGE: Machine-generated text detection in the wild. In *Proceedings of ACL*, 2024.
- [LZZ<sup>+</sup>23] Yikang Liu, Ziyin Zhang, Wanyang Zhang, et al. Argugpt: Evaluating, understanding and identifying argumentative essays generated by gpt models. In *arXiv preprint arXiv:2304.07666*, 2023.
- [M<sup>+</sup>23] Eric Mitchell et al. Detectgpt: Zero-shot machine-generated text detection using probability curvature. In *ICML*, 2023.
- [MKMS24] Dominik Macko, Jakub Kopal, Robert Moro, and Ivan Srba. Multisocial: Multilingual benchmark of machine-generated text detection of social-media texts. *arXiv preprint arXiv:2406.12549*, 2024.
- [MMU<sup>+</sup>23] Dominik Macko, Robert Moro, Adaku Uchendu, Jason Samuel Lucas, Michiharu Yamashita, Matúš Pikuliak, Ivan Srba, Thai Le, Dongwon Lee, Jakub

- Simko, et al. Multitude: Large-scale multilingual machine-generated text detection benchmark. *arXiv preprint arXiv:2310.13606*, 2023.
- [Ope23] OpenAI. Gpt-4 technical report. 2023.
- [PKKH25] Shinwoo Park, Shubin Kim, Do-Kyung Kim, and Yo-Sub Han. Katfishnet: Detecting llm-generated korean text through linguistic feature analysis. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 21189–21222, 2025.
- [Qui23] QuillBot. Quillbot: Ai-powered paraphrasing tool, 2023.
- [SGFS<sup>+</sup>23] Areg Mikael Sarvazyan, José Angel González, Marc Franco-Salvador, Francisco Rangel, Berta Chulvi, and Paolo Rosso. Overview of autextification at iberlef 2023: Detection and attribution of machine-generated text in multiple domains. *arXiv preprint arXiv:2309.11285*, 2023.
- [SMC<sup>+</sup>22a] Tatiana Shamardina, Vladislav Mikhailov, Daniil Chernianskii, Alena Fenogenova, Marat Saidov, Anastasiya Valeeva, Tatiana Shavrina, Ivan Smurov, Elena Tutubalina, and Ekaterina Artemova. Findings of the ru-atd shared task 2022 on artificial text detection in russian. *arXiv preprint arXiv:2206.01583*, 2022.
- [SMC<sup>+</sup>22b] Tatiana Shamardina, Vladislav Mikhailov, Daniil Chernianskii, Alena Fenogenova, Marat Saidov, Anastasiya Valeeva, Tatiana Shavrina, Ivan Smurov, Elena Tutubalina, and Ekaterina Artemova. Findings of the ru-atd shared task 2022 on artificial text detection in russian. *arXiv preprint arXiv:2206.01583*, 2022.
- [SWD<sup>+</sup>17] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

- [SWMH23] Zhenpeng Su, Xing Wu, Guangyuan Ma, and Songlin Hu. HC3 Plus: A semantic-invariant human ChatGPT comparison corpus. *arXiv preprint*, 2023.
- [SWX<sup>+</sup>24] Zhenqiang Shao, Peiyi Wang, Runxin Xu, Junxiao Song, Yiming Wang, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [SZWN23] Jinyan Su, Terry Yue Zhuo, Di Wang, and Preslav Nakov. Detectllm: Leveraging log rank information for zero-shot detection of machine-generated text. In *Findings of EMNLP*, 2023.
- [W<sup>+</sup>24] Yuxia Wang et al. M4: Multi-generator, multi-domain, and multi-lingual black-box machine-generated text detection. In *Proceedings of EACL*, 2024.
- [WFH<sup>+</sup>24] Yichen Wang, Shangbin Feng, Abe Bohan Hou, et al. Stumbling blocks: Stress testing the robustness of machine-generated text detectors under attacks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [WMI<sup>+</sup>24a] Yuxia Wang, Jonibek Mansurov, Petar Ivanov, Jinyan Su, Artem Shelmanov, Akim Tsvigun, Osama Mohammed Afzal, Tarek Mahmoud, Giovanni Puccetti, Thomas Arnold, et al. Semeval-2024 task 8: Multidomain, multi-model and multilingual machine-generated text detection. *arXiv preprint arXiv:2404.14183*, 2024.
- [WMI<sup>+</sup>24b] Yuxia Wang, Jonibek Mansurov, Petar Ivanov, Jinyan Su, Artem Shelmanov, Akim Tsvigun, Osama Mohammed Afzal, Tarek Mahmoud, Giovanni Puccetti, Thomas Arnold, et al. M4gt-bench: Evaluation benchmark for black-box machine-generated text detection. *arXiv preprint arXiv:2402.11175*, 2024.
- [WSM<sup>+</sup>25] Yuxia Wang, Artem Shelmanov, Jonibek Mansurov, Akim Tsvigun, Vladislav Mikhailov, Rui Xing, Zhuohan Xie, Jiahui Geng, Giovanni Puccetti, Eka-

- terina Artemova, et al. Genai content detection task 1: English and multilingual machine-generated text detection: Ai vs. human. *arXiv preprint arXiv:2501.11012*, 2025.
- [WYZ<sup>+</sup>23a] Junchao Wu, Shu Yang, Runzhe Zhan, et al. A survey on llm-generated text detection: Necessity, methods, and future directions. *arXiv preprint arXiv:2310.14724*, 2023.
- [WYZ<sup>+</sup>23b] Junchao Wu, Shu Yang, Runzhe Zhan, Yulin Yuan, Derek F. Wong, and Lidia S. Chao. A survey on LLM-generated text detection: Necessity, methods, and future directions. *arXiv preprint arXiv:2310.14724*, 2023.
- [X<sup>+</sup>21] Linting Xue et al. mt5: A massively multilingual pre-trained text-to-text transformer. *NAACL*, 2021.
- [XH77] Tsung-Yi Ho Xiaomeng Hu, Pin-Yu Chen. Radar: Robust ai-text detection via adversarial learning”. 1977. in ‘Steps into Computational Geometry’.